



Exercises session 8

Space Complexity

1 Definitions

Definition 1 • For a k -tape deterministic TM with input/output the space complexity function $s_M(n)$ is defined as the maximum number of cells on work tapes that M scans when processing any input of length n .

- For a non-deterministic machine this is the maximum over all branches of computation and all inputs of length n .
- In both cases we say that the machine M runs in space $s_M(n)$, or that $s_M(n)$ is the space complexity of M .

Definition 2 (Space complexity) Let $f : \mathbb{N} \rightarrow \mathbb{N}$ be a function. We define:

$SPACE(f(n)) = \{L \mid L \text{ is decided by an } O(f(n))$
space deterministic Turing machine with input/output}.

$NSPACE(f(n)) = \{L \mid L \text{ is decided by an } O(f(n))$
space non-deterministic Turing machine with input/output}.

2 Session exercises

1. Consider the following algorithm

```
def sumList(list):  
    sum = 0  
    for item in list:  
        sum += item  
    return sum
```

Indicate both the time and space complexity for this algorithm.

2. Consider the following recursive function

```
def recFunction(n):  
    if n <= 1:  
        return n  
    return recFunction(n-1) + recFunction(n-2)
```

3. Prove that the following problem is in PSPACE

$$EQ_{REG} = \{\langle A, B \rangle : A \text{ and } B \text{ are equivalent regular expressions}\}$$

4. Show that PSPACE is closed under union, complement and kleene-star.

P1.

- time complexity

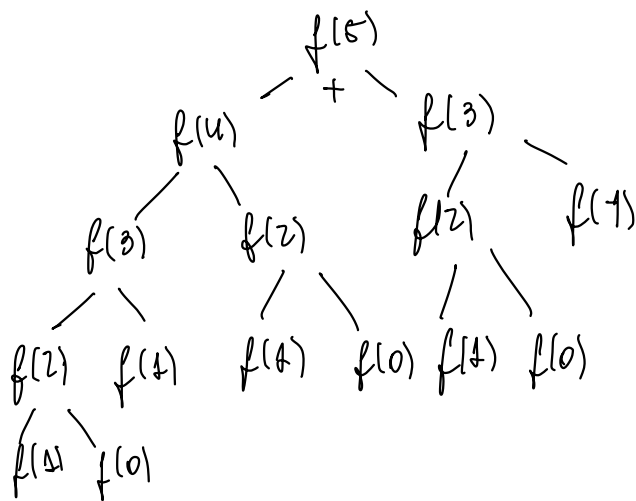
We need to go over the list, so the time complexity will be linear $O(n)$.

- space complexity

We only need to keep track of the current sum, so the space needed does not escalate w.r.t the size of the input $O(1)$ space complexity.

P2. this is a recursive way of constructing a Fibonacci series

Let's look at the recursion, see the recursion tree



the recursion looks like this,

$$T(n) = T(n-1) + T(n-2)$$

• depth $\rightarrow \Theta(n)$ (# return stat. before here)

• breadth $\rightarrow \Theta(2^n)$ (# rec. calls)

$\therefore$ space $\Theta(n)$
time $\Theta(2^n)$

storage unit for
 $f(0), f(1), \dots, f(n)$

P3,

$$\text{PSPACE} = \bigcup_{k \in \mathbb{N}} \text{SPACE}(n^k)$$

For a language A ,

$$\bar{A} \in \text{NPSPACE}$$

$$\Rightarrow \bar{A} \in \text{PSPACE}$$

$$\Rightarrow A \in \text{PSPACE}.$$

$\therefore$ it suffices to show that $\overline{\text{EQ}_{\text{NFA}}} \in \text{NPSPACE}$
to conclude that $\text{EQ}_{\text{NFA}} \in \text{PSPACE}.$

• let's define a NTM for $\overline{\text{EQ}_{\text{NFA}}}$

$$\text{INPUT} = \langle R, S \rangle$$

• Construct NFA for R , and S

$$N_R = (Q_R, \Sigma, \delta_R, q_R, A_R)$$

$$N_S = (Q_S, \Sigma, \delta_S, q_S, A_S)$$

$$m_R = \{q_R\}$$

$$m_S = \{q_S\}$$

- $NPSPACE = CONSPACE$

Lemma: $\forall$ deterministic class C

$$C = coC.$$

if $L \in C$, then take DTM all

$$s.t. L(1^n) = L$$

it suffices to change accepting states
flip

with rejecting ones, while keeping
the space/time.

now, from Savitch theorem

$$NPSPACE = PSPACE.$$

$$\Rightarrow CONSPACE$$

$$= coPSPACE$$

$$= PSPACE$$

$$= NPSPACE$$